PHASE 1
Research Foundation + Core Evidence Reconciliation Engine

1.1 Documentation Understanding
1.2 Research Specification Freeze
1.3 Evidence Model + Input Schema
1.4 PostgreSQL Foundation
1.5 Canonical Normalization
1.6 Entity Resolution
1.7 Synthetic Lifecycle Generator
1.8 Layer 1 Data Integrity Engine
1.9 Layer 2 Statistical Anomaly Engine
1.10 Baseline Systems
1.11 TriNetra Multi-Source Reconciliation
1.12 Experimental Framework
1.13 Evaluation + Metrics
1.14 Ablation + Error Analysis
1.15 Research Results Package
1.16 Phase 1 Final Validation

TriNetra AI
PHASE 1 MASTER IMPLEMENTATION README
Research Foundation + Core Evidence Reconciliation Engine

ROLE

You are the Lead Systems Architect, Research Engineer, and Academic Research Assistant for TriNetra AI.

Your responsibility is to build Phase 1 of TriNetra in a research-first manner.

Do NOT immediately start coding.

The first responsibility is to understand the complete project documentation and establish a validated implementation baseline.

PHASE 1 OBJECTIVE

Phase 1 is the foundation of the entire TriNetra project.

Its purpose is to:

1. Understand the complete TriNetra documentation.
2. Freeze the research problem and hypothesis.
3. Define the evidence model.
4. Define the machine-readable input format.
5. Build the PostgreSQL research foundation.

6. Generate controlled synthetic evidence.
7. Detect deterministic inconsistencies.
8. Detect statistical anomalies.
9. Compare single-source baselines against multi-source reconciliation.
10. Produce reproducible research metrics.
11. Produce the first experimental results that can support the research paper.

Phase 1 must NOT attempt to build the complete production platform.

PHASE 1 RULE

Complete each subphase independently.

At the end of EVERY subphase:

1. Run validation.
2. Report what was completed.
3. Report what failed.
4. Report what remains.
5. Verify against the project documentation.
6. Do not continue automatically if validation fails.

Only continue to the next subphase after the current subphase passes its validation gate.

SUBPHASE 1.1
DOCUMENTATION UNDERSTANDING

Before Coding

Read all provided TriNetra project documentation.

This includes:

- Product Requirements Document
- Technical Requirements Document
- Software Requirements Specification
- Research specification
- Research README
- Literature validation material
- Complaint research
- Existing architecture documents
- Any additional project documentation available in the workspace

Do not assume the documentation is internally correct.

Do not silently invent missing requirements.

Do not silently replace the project's terminology.

Create

Create:

docs/PHASE_1_DOCUMENTATION_UNDERSTANDING.md

Include:

1. Project Purpose

Explain what TriNetra is.

2. Core Problem

Explain the problem TriNetra is attempting to solve.

3. Research Hypothesis

State the current research hypothesis exactly as supported by the documentation.

4. Stakeholders

Document:

- Marketplace
- Seller
- Warehouse / Fulfillment
- Carrier
- Delivery
- Return Center
- Customer

5. Evidence Sources

Document all evidence sources.

6. Core Reconciliation Problem

Explain what "reconciliation" means in this project.

7. Current Technical Architecture

Document the planned architecture.

8. Phase 1 Scope

Clearly separate:

IN SCOPE

from:

OUT OF SCOPE

9. Research Constraints

Document:

- PostgreSQL
- reproducibility
- deterministic results
- evidence provenance
- no fabricated evidence
- no automatic fraud accusation

10. Existing Assumptions

List assumptions from the documentation separately from experimentally validated facts.

Critical Requirement

Do not start implementation after reading the documents.

First produce:

docs/PHASE_1_DOCUMENTATION_UNDERSTANDING.md

Then STOP.

VALIDATION GATE 1

Phase 1.1 passes only if:

[ ] Every provided document has been reviewed.

[ ] Core project objective is documented.

[ ] Research hypothesis is documented.

[ ] Stakeholders are documented.

[ ] Evidence sources are documented.

[ ] Phase 1 scope is documented.

[ ] Unknown assumptions are explicitly listed.

[ ] No undocumented major feature was invented.

If any condition fails:

STOP and report the failure.

SUBPHASE 1.2
RESEARCH SPECIFICATION FREEZE

After Phase 1.1 is approved, define:

docs/PHASE_1_RESEARCH_SPECIFICATION.md

Include:

Primary Research Question

Can multi-source, cross-organizational evidence reconciliation reduce false negatives compared with single-source verification when identifying inconsistencies in e-commerce product lifecycles?

Supporting Questions

1. Can independent evidence sources be normalized?
2. Can the same product be resolved across organizations?
3. Can deterministic conflicts be detected?
4. Can statistical anomalies be detected?
5. Does combining evidence reduce false negatives?
6. Which evidence sources contribute most?
7. How does missing evidence affect performance?

Ground Rules

TriNetra must:

- detect observable inconsistencies
- preserve evidence provenance
- avoid inventing evidence
- avoid automatic guilt claims
- produce reproducible results

VALIDATION GATE 2

Verify that:

[ ] Research question is frozen.

[ ] Supporting questions are frozen.

[ ] Evaluation targets are defined.

[ ] No unsupported performance target is treated as a guaranteed result.

STOP if validation fails.

SUBPHASE 1.3
EVIDENCE MODEL AND INPUT FORMAT

The Phase 1 evidence input format will be JSON.

The external input represents stakeholder evidence.

Example structure:

```
{
  "order": {
    "order_id": "ORD-1001",
    "product_id": "PROD-001",
    "sku": "TS-204",
    "size": "XL",
    "color": "BLACK",
    "timestamp": "2026-08-23T10:00:00Z"
  },
  "evidence": [
    {
      "source": "SELLER",
      "organization_id": "SELLER-01",
      "event_type": "PRODUCT_DISPATCH",
      "timestamp": "2026-08-23T10:30:00Z",
      "attributes": {
        "sku": "TS-204",
        "size": "XL",
        "color": "BLACK"
      }
    }
  ]
```

}

The system must preserve:

- original value
- normalized value
- source
- organization
- event
- timestamp
- provenance

Do not reduce all evidence to one JSON blob in the database.

VALIDATION GATE 3

Verify:

[ ] JSON structure is defined.

[ ] Required fields are defined.

[ ] Optional fields are defined.

[ ] Provenance requirements are defined.

[ ] Canonical normalization rules are defined.

[ ] Example valid evidence exists.

[ ] Example invalid evidence exists.

STOP.

SUBPHASE 1.4
POSTGRESQL FOUNDATION

Create exactly:

entity_resolution
evidence_record
evidence_attribute

The schema must support:

- canonical entity IDs

- organization-specific IDs
- source systems
- event timestamps
- evidence types
- original values
- normalized values
- units
- provenance
- confidence

Do not build application APIs.

VALIDATION GATE 4

Run:

- schema creation
- insertion tests
- foreign-key tests
- constraint tests
- duplicate tests
- normalization storage tests

Generate:

docs/PHASE_1_DATABASE_VALIDATION.md

STOP if any schema test fails.

SUBPHASE 1.5
CANONICAL NORMALIZATION

Implement normalization for:

- SKU
- Product ID
- size
- color
- weight
- timestamp
- batch
- serial number

Example:

"Extra Large"

"extra-large"
"XL"

→ XL

Example:

"0.5kg"
"500g"

→ 500 grams

Never overwrite the original value.

VALIDATION GATE 5

Create normalization test cases.

Minimum:

10 valid normalization cases
10 invalid cases
5 edge cases

All must pass.

STOP if any fail.

SUBPHASE 1.6
ENTITY RESOLUTION

Resolve organization-specific identifiers to canonical entities.

Example:

Seller:
SELLER-991

Warehouse:
WH-20491

Carrier:
PKG-8821

Canonical:
PROD-001

Record:

- source ID
- canonical ID
- organization
- resolution method
- resolution confidence

Initially use deterministic resolution.

VALIDATION GATE 6

Test:

- correct mapping
- duplicate mapping
- wrong mapping
- missing mapping
- ambiguous mapping

Produce a validation report.

STOP if failures exist.

SUBPHASE 1.7
SYNTHETIC LIFECYCLE GENERATOR

Generate:

ORDER
→ SELLER
→ WAREHOUSE
→ PACKAGING
→ CARRIER
→ DELIVERY
→ RETURN

Generate normal and anomalous lifecycles.

Required conflicts:

- identity
- SKU
- variant
- weight

- timestamp
- missing evidence
- duplicate evidence
- multi-source conflict

Every sample MUST contain ground truth.

Ground truth must exist independently of the detection engine.

VALIDATION GATE 7

Verify:

[ ] All lifecycle stages can be generated.

[ ] Normal cases exist.

[ ] Every conflict type exists.

[ ] Ground truth exists.

[ ] No label is generated from detector output.

[ ] Random seed is recorded.

STOP if validation fails.

SUBPHASE 1.8
LAYER 1 DATA INTEGRITY

Implement deterministic checks:

- identity
- SKU
- variant
- timestamps
- units
- missing evidence
- duplicate evidence

Output structured conflicts.

Do NOT call them fraud.

Example:

```
{
  "conflict_type": "SKU_CONFLICT",
  "source_a": "SELLER",
  "value_a": "TS204",
  "source_b": "WAREHOUSE",
  "value_b": "TS205"
}
```

VALIDATION GATE 8

Create known-good and known-bad test cases.

Every expected conflict must be detected.

Every normal case must remain normal.

Measure false positives even for deterministic checks.

STOP on failure.

SUBPHASE 1.9
LAYER 2 STATISTICAL ANOMALY DETECTION

Implement weight anomaly detection.

Do not blindly use a fixed 5% rule.

Model:

- product weight
- packaging variance
- measurement variance
- expected operational variation

Possible methods:

- z-score
- robust z-score
- MAD
- IQR

Choose one based on the generated distribution.

Document why.

VALIDATION GATE 9

Evaluate:

- normal weights
- small deviations
- major deviations
- outliers
- different product categories
- different packaging conditions

Measure:

- false positives
- false negatives

STOP if results are not reproducible.

SUBPHASE 1.10
BASELINES

Implement:

Baseline 1:
Identity-only.

Baseline 2:
Weight-only.

Baseline 3:
Timeline-only.

Then implement:

TriNetra:
Multi-source reconciliation.

Each must produce predictions independently.

VALIDATION GATE 10

Verify:

[ ] All baselines run independently.

[ ] No baseline receives TriNetra output.

[ ] Ground truth remains independent.

[ ] All experiments use identical test sets.

STOP on failure.

SUBPHASE 1.11
TRINETRA MULTI-SOURCE RECONCILIATION

Combine:

- identity
- attribute consistency
- weight
- timeline
- evidence completeness

Output:

CONSISTENT
INCONSISTENT
INCONCLUSIVE

The engine must explain which evidence sources caused the inconsistency.

Do not use an LLM.

Do not generate natural-language fraud claims.

VALIDATION GATE 11

Test:

- normal
- single conflict
- multiple conflicts
- missing evidence
- conflicting evidence
- noisy evidence

Verify reproducibility.

STOP on failure.

SUBPHASE 1.12
EXPERIMENTAL FRAMEWORK

Create controlled experiments.

Dimensions:

Evidence completeness

100%
75%
50%
25%

Conflict difficulty

Low
Medium
High

Conflict type

Identity
SKU
Variant
Weight
Temporal
Missing evidence
Multiple conflicts

Run each experiment with recorded:

- random seed
- configuration
- dataset version
- algorithm version

VALIDATION GATE 12

Verify experiments are reproducible.

Running the same experiment twice must produce identical results when the seed and configuration are unchanged.

SUBPHASE 1.13
EVALUATION

Calculate:

Precision
Recall
F1
False Positive Rate
False Negative Rate
False Negative Reduction

Primary metric:

False Negative Reduction.

Formula:

(Baseline FN - TriNetra FN) / Baseline FN

Calculate results:

- overall
- by conflict type
- by evidence completeness
- by difficulty

VALIDATION GATE 13

Verify:

[ ] Metrics are mathematically correct.

[ ] Ground truth is independent.

[ ] No test leakage exists.

[ ] Results are reproducible.

[ ] Results are saved.

STOP if any condition fails.

SUBPHASE 1.14
ABLATION AND ERROR ANALYSIS

Run:

A. Identity only

B. Identity + Weight

C. Identity + Timeline

D. Identity + Weight + Timeline

E. Full TriNetra

Determine which evidence types actually contribute.

Analyze:

- false positives
- false negatives
- hard cases
- missing evidence
- contradictory evidence

VALIDATION GATE 14

Produce:

ablation_results.csv
error_analysis.csv

The report must identify:

- strongest evidence source
- weakest evidence source
- failure cases
- unresolved limitations

Do not selectively report only positive results.

SUBPHASE 1.15
RESEARCH RESULTS PACKAGE

Generate:

phase-1/results/

metrics.csv

confusion_matrix.csv
conflict_metrics.csv
ablation_results.csv
error_analysis.csv
experiment_config.json
experiment_summary.md

Generate publication-quality figures:

- baseline comparison
- precision/recall
- false-negative reduction
- conflict-type performance
- evidence-completeness performance
- ablation results

Also create:

docs/PHASE_1_RESEARCH_RESULTS.md

Include:

1. Dataset generation
2. Methodology
3. Baselines
4. Experimental setup
5. Results
6. Error analysis
7. Limitations
8. Research implications

SUBPHASE 1.16
FINAL PHASE 1 VALIDATION

This is mandatory.

Do NOT declare Phase 1 complete merely because the code runs.

Create:

docs/PHASE_1_FINAL_VALIDATION.md

Check every requirement:

Research

[ ] Research question implemented.

[ ] Hypothesis testable.

[ ] Baselines implemented.

[ ] Experimental protocol documented.

Data

[ ] Synthetic generator works.

[ ] Ground truth independent.

[ ] Evidence normalization works.

[ ] Entity resolution works.

Database

[ ] PostgreSQL schema works.

[ ] Integrity constraints pass.

[ ] Evidence provenance preserved.

Engine

[ ] Deterministic conflicts detected.

[ ] Statistical anomalies detected.

[ ] Multi-source reconciliation works.

Evaluation

[ ] Precision calculated.

[ ] Recall calculated.

[ ] F1 calculated.

[ ] False-positive rate calculated.

[ ] False-negative rate calculated.

[ ] False-negative reduction calculated.

[ ] Ablation completed.

[ ] Error analysis completed.

Reproducibility

[ ] Seed stored.

[ ] Experiment configuration stored.

[ ] Dataset version stored.

[ ] Results reproducible.

Research Integrity

[ ] No fabricated metrics.

[ ] No fabricated evidence.

[ ] No automatic fraud accusation.

[ ] Limitations documented.

FINAL PHASE 1 DECISION

Return exactly one of:

PHASE_1_COMPLETE

or

PHASE_1_INCOMPLETE

If INCOMPLETE:

List every failed validation item.

Do NOT begin Phase 2.

If COMPLETE:

Provide:

1. Final architecture
2. Final schema
3. Dataset statistics
4. Baseline results
5. TriNetra results
6. False-negative reduction
7. Ablation findings
8. Error analysis

9. Limitations
10. Recommended Phase 2 objectives

Then STOP.

Wait for explicit authorization before Phase 2.